Metasploitable 2

Tuesday Notes — VSFTPD 2.3.4 Backdoor & Metasploit Exploitation

Part 1 — Understanding the VSFTPD 2.3.4 Backdoor
Part 2 — Metasploit Basics & VSFTPD Exploitation

PART 1

Understanding the VSFTPD 2.3.4 Backdoor

1. Why Version Detection Matters

During enumeration, the following was identified:

- **Port:** 21
- **Service:** FTP
- **Software:** vsftpd 2.3.4

This is far more valuable than simply knowing that an FTP service is running. Knowing the exact software and version allows targeted research into known vulnerabilities.

WITHOUT VERSION DETECTION	WITH VERSION DETECTION
• "An FTP server is running."	• "vsftpd 2.3.4 is running." → can now research this specific version

2. What is FTP?

FTP (File Transfer Protocol) is a protocol used to transfer files between computers. A protocol is simply a set of agreed rules that both computers follow to communicate.

Common FTP commands:

- USER
- PASS
- LIST
- RETR
- STOR

3. What is a Daemon?

A daemon is a program that runs continuously in the background to provide a service. It usually waits for work instead of interacting directly with a user.

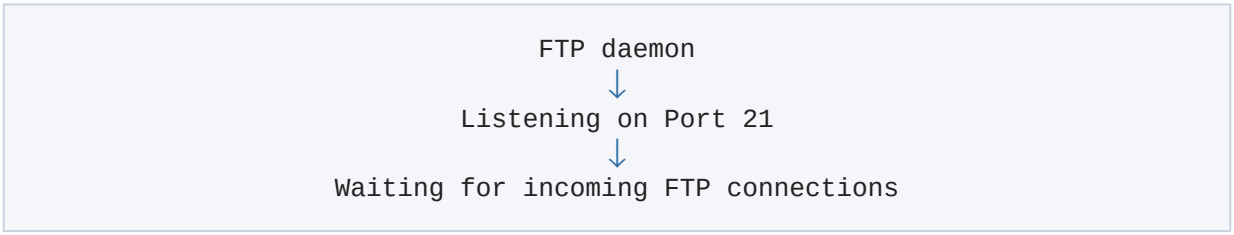
Examples:

- FTP daemon
- HTTP daemon
- SSH daemon

Many Linux server programs end with "d", meaning daemon — e.g. **httpd** = HTTP daemon, **sshd** = SSH daemon. **vsftpd** is an FTP server daemon.

4. Listening on a Port

A daemon often listens on a network port. Listening means: *"I am waiting for someone to connect."*



When a client disconnects, the daemon simply returns to waiting for the next connection.

5. Network Daemons vs Other Daemons

Not every daemon listens on a network.

NETWORK DAEMONS	OTHER DAEMONS
• FTP daemon	• Logging daemon
• HTTP daemon	• Cron scheduler
• SSH daemon	• Printing daemon

A daemon simply provides a background service.

6. Why VSFTPD 2.3.4 Became Famous

The original developers did **not** intentionally create a backdoor. Instead:

- The official download server was compromised.
- An attacker replaced the legitimate vsftpd 2.3.4 archive with a maliciously modified copy.
- Anyone who downloaded that modified archive installed a backdoored FTP server.

This is called a **software supply chain attack** — it targeted the software distribution process rather than exploiting users individually.

7. Original vs Trojanized VSFTPD 2.3.4

There were effectively two copies of version 2.3.4:

ORIGINAL VERSION	TROJANIZED VERSION
• Written by the developers	• Modified by an attacker
• Safe	• Contained a hidden backdoor

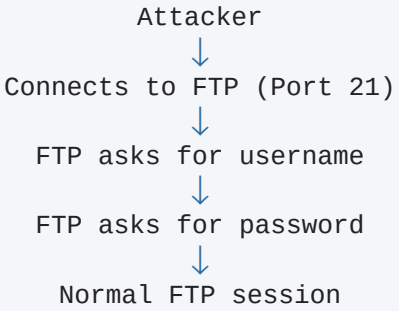
Metasploitable 2 intentionally includes the trojanized version for learning purposes.

8. What is a Backdoor?

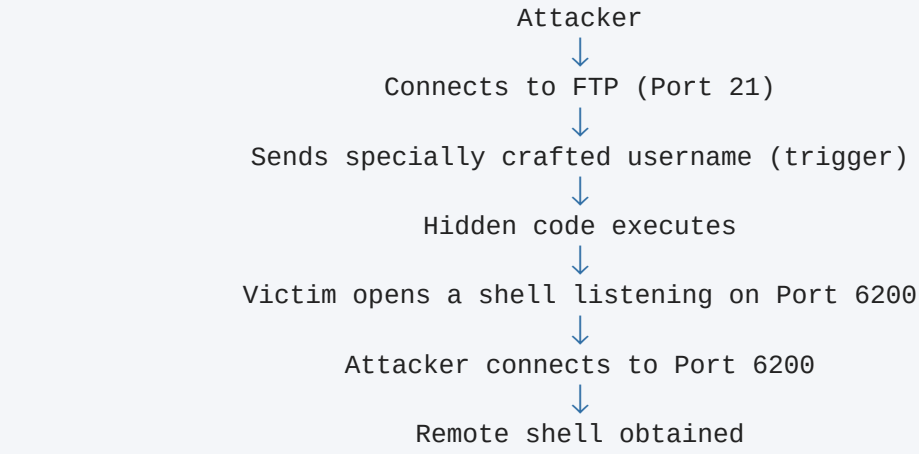
A backdoor is hidden functionality intentionally added to software that allows unauthorized access when a secret trigger is used. Unlike many exploits, this is **not a programming bug** — the malicious functionality was deliberately inserted into the software.

9. How the Backdoor Works

NORMAL OPERATION



BACKDOORED OPERATION



10. What is the Trigger?

The trigger is a specially crafted username. The FTP daemon checks for this special input — if detected, the hidden malicious code runs; otherwise, FTP behaves normally.

11. Does the Victim Connect Back?

No. The victim does not initiate a connection to the attacker. Instead:

- The attacker connects to FTP (Port 21).
- The trigger is sent.
- The victim opens a new listening port (6200).
- The attacker makes a second connection to Port 6200.

Both connections are initiated by the attacker.

12. Why Open Another Port?

FTP and a shell use different protocols. FTP understands FTP commands; a Linux shell understands operating system commands.

FTP	SHELL
• USER	• ls
• PASS	• pwd
• LIST	• whoami
	• cat

Instead of mixing these protocols together, the backdoor opens a separate listening port for the shell.

13. Bind Shell

The shell created by the backdoor listens on the victim's machine, and the attacker connects to that listening port. This is called a **bind shell** because the shell is bound to a listening port on the victim. A **reverse shell** is different — it causes the victim to initiate the connection back to the attacker.

KEY TAKEAWAYS

- Enumeration identifies services and exact software versions.
- Version detection is essential because vulnerabilities are often version-specific.
- A daemon is simply a background program that provides a service.
- Many daemons listen on network ports, but not all do.
- vsftpd is an FTP daemon.
- The famous vsftpd 2.3.4 incident was a software supply chain attack, not a coding bug.
- Metasploitable 2 intentionally contains the backdoored version for training.
- The attacker triggers the backdoor through FTP.
- The victim opens a new listening port for a shell.
- The attacker then connects to that shell using a second connection.

PART 2

Metasploit Basics & VSFTPD Exploitation

Goal

The objective of this session was **not simply to gain a shell**, but to understand every stage of a real exploitation workflow.

Focus areas:

- How to select a target from enumeration
- How to research a vulnerability
- How Metasploit is organized
- The difference between an exploit and a payload
- What happens internally during exploitation
- How defenders could detect and prevent the attack

1. Choosing the Target

From Monday's enumeration, several vulnerable services were identified. The chosen target was:

- **Service:** FTP
- **Port:** 21/TCP
- **Software:** vsftpd 2.3.4

Reason for choosing it:

- Famous real-world vulnerability
- Reliable exploit
- Excellent for learning Metasploit fundamentals
- Simple exploitation chain

Important lesson: Enumeration determines what is worth attacking. Exploitation begins only after identifying software and versions.

2. Understanding VSFTPD

VSFTPD stands for **Very Secure FTP Daemon**.

- **FTP** = File Transfer Protocol
- **Daemon** = Background service waiting for network connections

The server continuously listens for incoming FTP connections on TCP port 21.

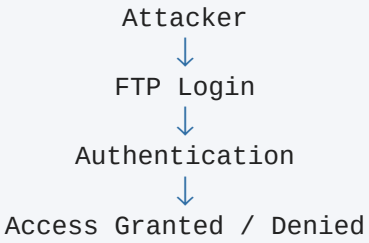
3. Why VSFTPD 2.3.4 Is Vulnerable

The vulnerability was **not** caused by a programming mistake. In 2011, attackers compromised the official VSFTPD download server and replaced the legitimate source archive with a maliciously modified version containing a hidden backdoor.

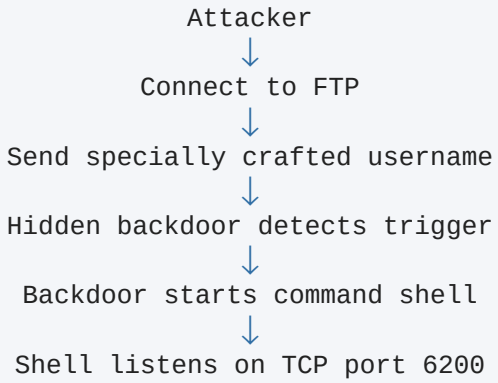
This is known as a **Software Supply Chain Compromise** — the software itself had been intentionally modified before users downloaded it. The original project was not designed this way.

4. How the Backdoor Works

NORMALLY



BACKDOORED VERSION



The login itself is not the goal — the specially crafted username activates the hidden code.

5. Opening Metasploit

```
msfconsole
```

Metasploit Framework is a collection of security modules. It contains:

- Exploit modules
- Payloads
- Auxiliary modules
- Post modules
- Encoders
- NOP generators

Today's focus: Exploit + Payload

6. Searching for the Exploit

```
search vsftpd
```

Result: `exploit/unix/ftp/vsftpd_234_backdoor`

Search results also display: module name, disclosure date, reliability rank, check support, and description.

Important lesson: Version information collected during enumeration determines which exploit module to search for.

7. Inspecting the Module

```
info exploit/unix/ftp/vsftpd_234_backdoor
```

Learned:

- **Platform:** Unix
- **Architecture:** cmd — the payload executes operating system commands rather than uploading machine code
- **Privilege:** Expected root access
- **Targets:** Automatic
- **Check:** No — the module cannot safely verify the vulnerability beforehand

8. Loading the Exploit

```
use exploit/unix/ftp/vsftpd_234_backdoor
```

Metasploit automatically selected: `cmd/unix/interact`

Important lesson: Loading an exploit does NOT attack the target — it simply loads the module into memory.

9. Exploit vs Payload

EXPLOIT — RESPONSIBLE FOR	PAYLOAD — RESPONSIBLE FOR
• Connecting to FTP • Sending the trigger • Activating the hidden backdoor	• Connecting to the shell that the backdoor already created

Important lesson: The payload did NOT create the shell. The compromised FTP server created the shell — the payload only interacted with it.

10. Configuring the Exploit

```
show options
```

Important fields:

- **RHOSTS** — Remote target IP address
- **RPORT** — Target port (default: 21)
- **CHOST** — Local client IP (usually unnecessary)
- **CPORT** — Local source port (normally auto-selected by the OS)
- **Proxies** — Optional proxy chain (unused in this lab)

```
set RHOSTS 10.174.253.161
```

11. Executing the Exploit

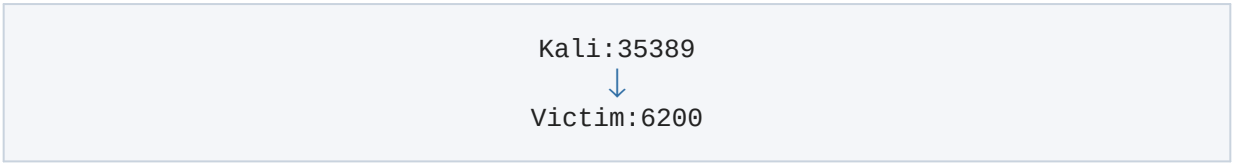
```
run
```

SEQUENCE PERFORMED INTERNALLY

12. Understanding the Output

- **Banner** — Confirmed: VSFTPD 2.3.4
- **USER** — FTP server responded normally; the trigger was hidden inside the username
- **Backdoor spawned** — A new shell process started
- **UID** — uid=0(root); root privileges obtained
- **Found shell** — Payload successfully connected
- **Session opened** — Metasploit created Session 1

Observed network connection:



Important lesson: Two TCP connections occurred.

CONNECTION 1	CONNECTION 2
• Kali → FTP Port 21	• Kali → TCP Port 6200
• Purpose: Trigger the backdoor	• Purpose: Connect to the shell

13. Session Interaction

```
whoami
```

Output: root — Commands execute with full administrative privileges.

```
pwd
```

Output: / — Current working directory is the root of the Linux filesystem.

```
ls
```

Displayed system directories including: bin, etc, home, root, usr, var.

Important lesson: Commands were executed on the victim machine, not on Kali.

14. Defender Perspective

Potential Indicators:

- FTP connection logs
- Suspicious username in authentication logs
- Unexpected listener on TCP port 6200
- Second TCP connection to port 6200

Detection Methods:

- FTP logs
- IDS signatures (Snort / Suricata)
- Network monitoring
- Process monitoring
- Listening port monitoring

Why Antivirus May Not Stop It: No malicious file was uploaded. The malicious code already existed inside the compromised FTP daemon — the attacker simply activated it.

Firewall Impact: If port 6200 were blocked, the payload could not connect, even if the trigger succeeded.

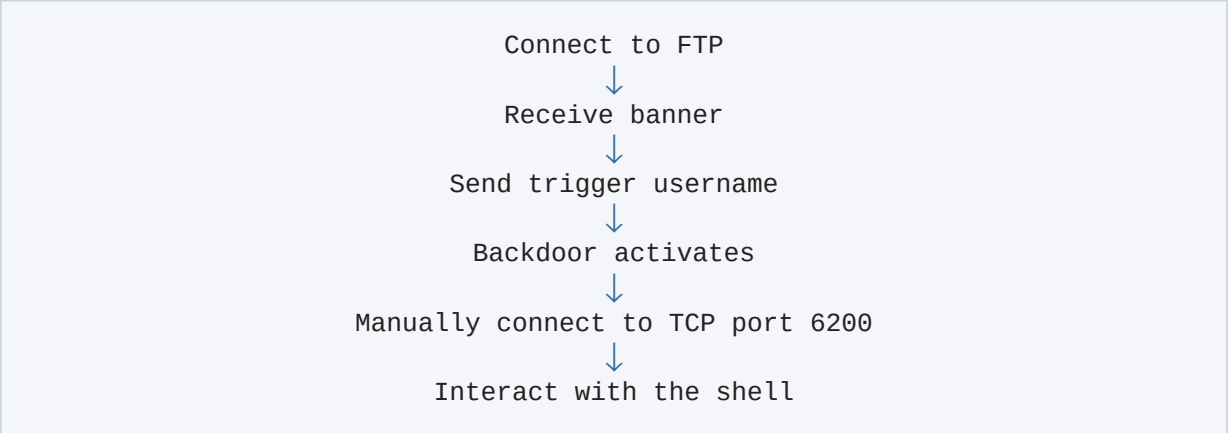
15. Remediation

The administrator should:

- Remove compromised VSFTPD version
- Install legitimate version
- Restrict FTP exposure
- Monitor authentication logs
- Monitor unexpected listening ports
- Follow the Principle of Least Privilege

16. Manual Exploitation Concept

Without Metasploit:



Metasploit simply automated these steps.

Commands Used

```
msfconsole
search vsftpd
info exploit/unix/ftp/vsftpd_234_backdoor
use exploit/unix/ftp/vsftpd_234_backdoor
show options
set RHOSTS 10.174.253.161
run
whoami
pwd
ls
```

KEY TAKEAWAYS

- ■ Enumeration determines exploitation targets.
- ■ Version detection is critical.
- ■ Not every exploit abuses a programming bug.
- ■ Supply chain compromises insert malicious code before installation.
- ■ Exploit and payload are separate components.
- ■ The exploit activates the vulnerability.
- ■ The payload uses the access gained.
- ■ Loading a module does not attack the target.
- ■ A Metasploit session represents an active connection to a compromised machine.
- ■ Manual exploitation follows the same logic as Metasploit.
- ■ Understanding the exploitation process is more valuable than memorizing commands.

Final Takeaway

The machine was **not compromised because Metasploit is powerful**. It was compromised because:

- Enumeration correctly identified the vulnerable service.
- The exact vulnerable version was discovered.
- The service was reachable.
- The installed software contained a hidden backdoor.
- The exploit triggered that backdoor.
- The payload simply connected to the shell that the backdoor created.

Metasploit automated the exploitation process, but understanding the sequence of events is what allows a penetration tester to work beyond automated tools.